

Challenge Assignment 2: Social network analysis and modeling

Gabriel Silva Cheinquer (4266390), Ege Basaran (8787433)

2026-04-06

Contents

Exercise 1	1
Question 1 (3 points)	4
Degree distribution and shortest path lengths	8
Question 2 (5 points) (<i>please submit your code along with your answer</i>)	8
Exercise 2	10
Question 3 (4 points)	10
Exercise 3	13
Question 4 (8 points)	13
Question 5 (8 points)	18

Exercise 1

Data preparation

In addition to the Facebook network you have used in the Learning-based exercise, you will use the Highschool network here.

Download the “Highschool_network_att.csv”, “Highschool_network_att.csv”, and “LargeFacebook_network_att.csv”, and “LargeFacebook_network_edge.csv” from BrightSpace. Import the .csv data into R and build the two networks.

```
library(igraph)
```

```
##
## Attaching package: 'igraph'

## The following objects are masked from 'package:stats':
##
##     decompose, spectrum
```

```
## The following object is masked from 'package:base':  
##  
##      union
```

```
library(RColorBrewer)  
library(visNetwork)
```

```
## Warning: package 'visNetwork' was built under R version 4.5.2
```

```
library(ggplot2)  
library(dplyr)
```

```
##  
## Attaching package: 'dplyr'
```

```
## The following objects are masked from 'package:igraph':  
##  
##      as_data_frame, groups, union
```

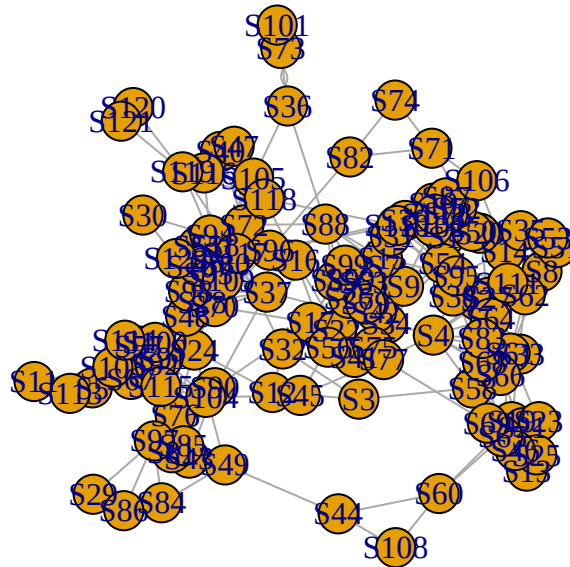
```
## The following objects are masked from 'package:stats':  
##  
##      filter, lag
```

```
## The following objects are masked from 'package:base':  
##  
##      intersect, setdiff, setequal, union
```

```
# Highschool network  
Highschool_edge <- read.csv("data/Highschool_network_edge.csv", header = FALSE)  
Highschool_att <- read.csv("data/Highschool_network_att.csv", header = TRUE)  
  
Highschool_nodes <- data.frame(  
  name = as.character(Highschool_att$NodeID),  
  gender = as.character(Highschool_att$Gender),  
  hall = as.character(Highschool_att$Hall)  
)  
  
Highschool_edges <- data.frame(  
  from = as.character(Highschool_edge[,1]),  
  to = as.character(Highschool_edge[,2])  
)  
  
Highschool <- graph_from_data_frame(  
  Highschool_edges,  
  directed = FALSE,  
  vertices = Highschool_nodes  
)  
  
summary(Highschool)
```

```
## IGRAPH 91e20f7 UN-- 122 396 --  
## + attr: name (v/c), gender (v/c), hall (v/c)
```

```
plot(Highschool)
```



```
# Large Facebook network
LargeFacebook_edge <- read.csv("data/LargeFacebook_network_edge.csv", header = FALSE)
LargeFacebook_att <- read.csv("data/LargeFacebook_network_att.csv", header = TRUE)

LargeFacebook_nodes <- data.frame(
  name = as.character(LargeFacebook_att$NodeID)
)

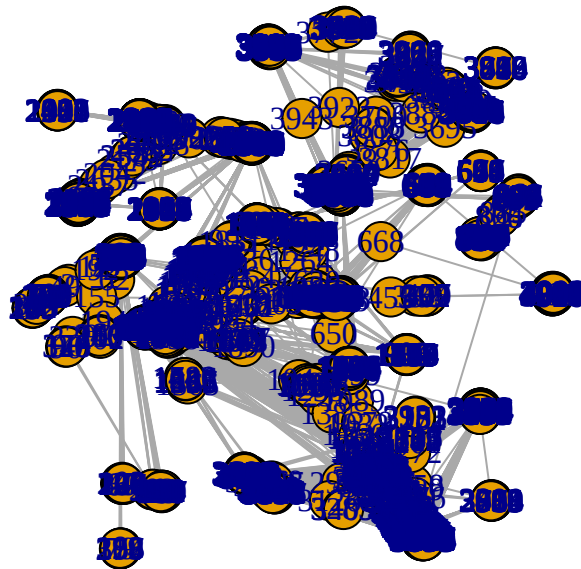
LargeFacebook_edges <- data.frame(
  from = as.character(LargeFacebook_edge[,1]),
  to   = as.character(LargeFacebook_edge[,2])
)

LargeFacebook <- graph_from_data_frame(
  LargeFacebook_edges,
  directed = FALSE,
  vertices = LargeFacebook_nodes
)

summary(LargeFacebook)
```

```
## IGRAPH 920da94 UN-- 4039 88234 --
## + attr: name (v/c)
```

```
plot(LargeFacebook)
```



Question 1 (3 points)

- For both the Highschool and Facebook networks, calculate the shortest path lengths between every pair of two nodes. How many percentage of nodes can be reached within 6 path lengths? Does “six degree of separation” apply to each network?
- Study the degree distribution of these two networks, are they similar? Then use degree distribution to explain the degree of separation you answered above.

You can calculate the shortest path lengths between every pair of two nodes using igraph package:

```
# Shortest path matrix Highschool

dist_highschool <- distances(
  Highschool,
  v = V(Highschool),
  to = V(Highschool),
```

```

mode = "all",
weights = NULL
)

# Convert to vector and remove self-distances
dist_high_vec <- dist_highschool[dist_highschool != 0]

# Percentage within 6 steps
mean(dist_high_vec <= 6) * 100

```

```
## [1] 98.56388
```

```

# Shortest path matrix LargeFacebook

dist_facebook <- distances(
  LargeFacebook,
  v = V(LargeFacebook),
  to = V(LargeFacebook),
  mode = "all",
  weights = NULL
)

dist_fb_vec <- dist_facebook[dist_facebook != 0]

mean(dist_fb_vec <= 6) * 100

```

```
## [1] 97.96999
```

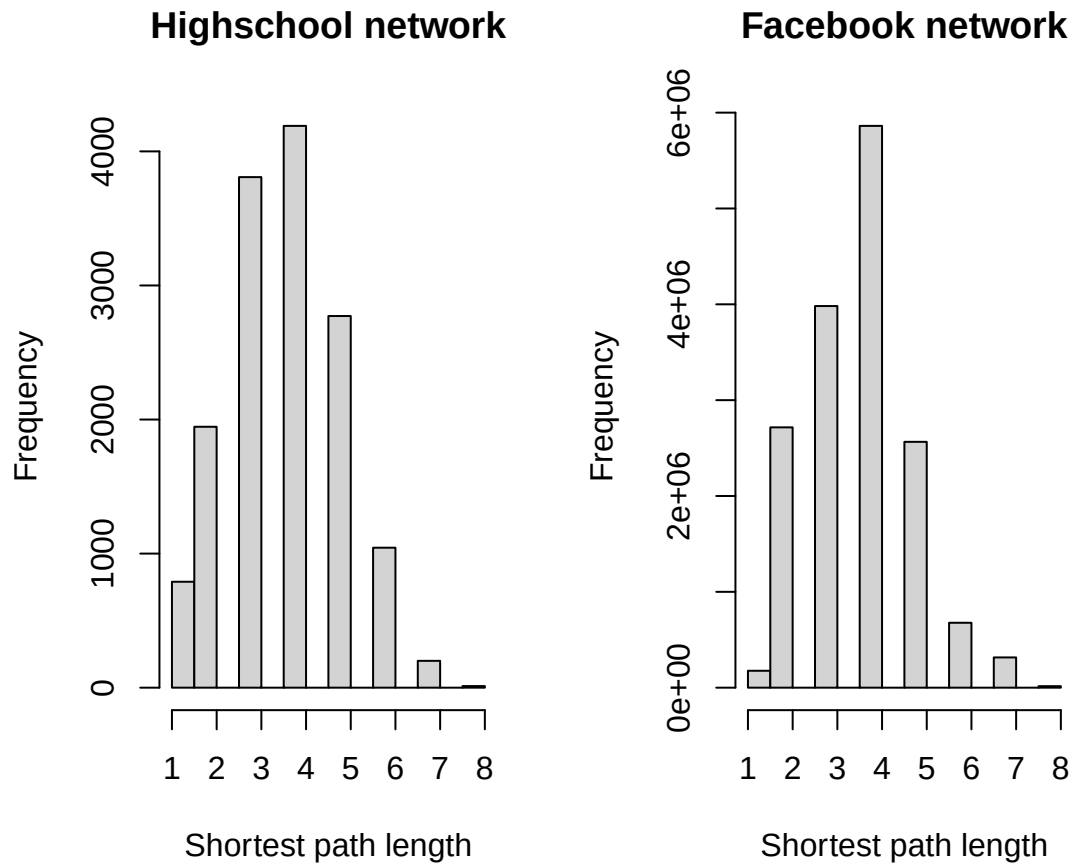
```

par(mfrow=c(1,2))

hist(dist_high_vec,
     breaks=20,
     main="Highschool network",
     xlab="Shortest path length")

hist(dist_fb_vec,
     breaks=20,
     main="Facebook network",
     xlab="Shortest path length")

```



We computed the proportion of node pairs that are reachable within 6 steps. For the Highschool network this comes out to about 98.6%, and for the Facebook network about 98.0%. In both cases the vast majority of shortest paths lie within 6 steps, so the six degrees of separation idea fits both networks well. In the histograms, most paths in the Highschool network sit around 3–5 steps, with very few beyond 6 and a maximum around 7–8. In the Facebook network, paths tend to be slightly shorter (around 3–4 steps), and almost every pair is reachable within 6 steps.

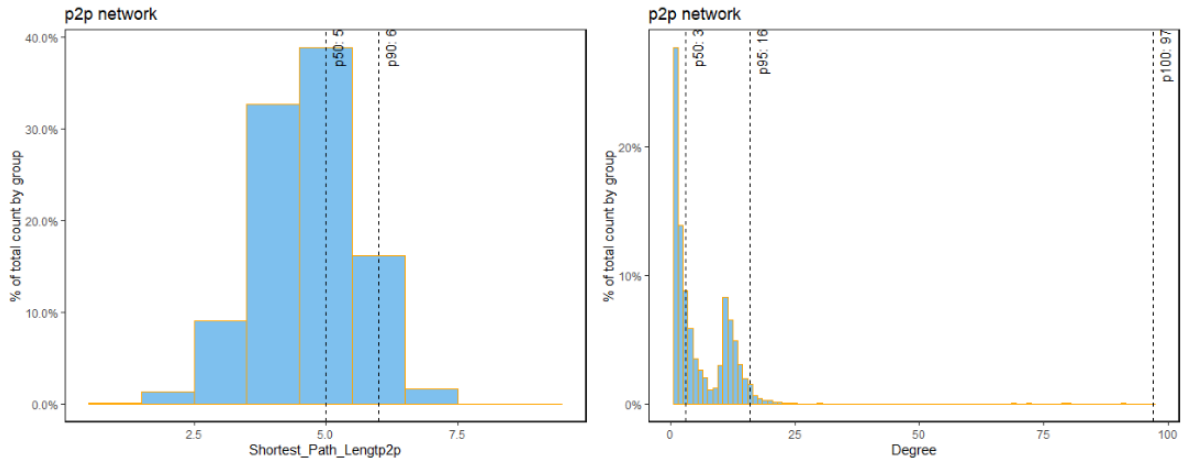
You can consider using a histogram with percentiles to study the shortest path lengths and the degree distribution:

```
deg_high <- degree(Highschool)
deg_fb <- degree(LargeFacebook)

par(mfrow=c(1,2))

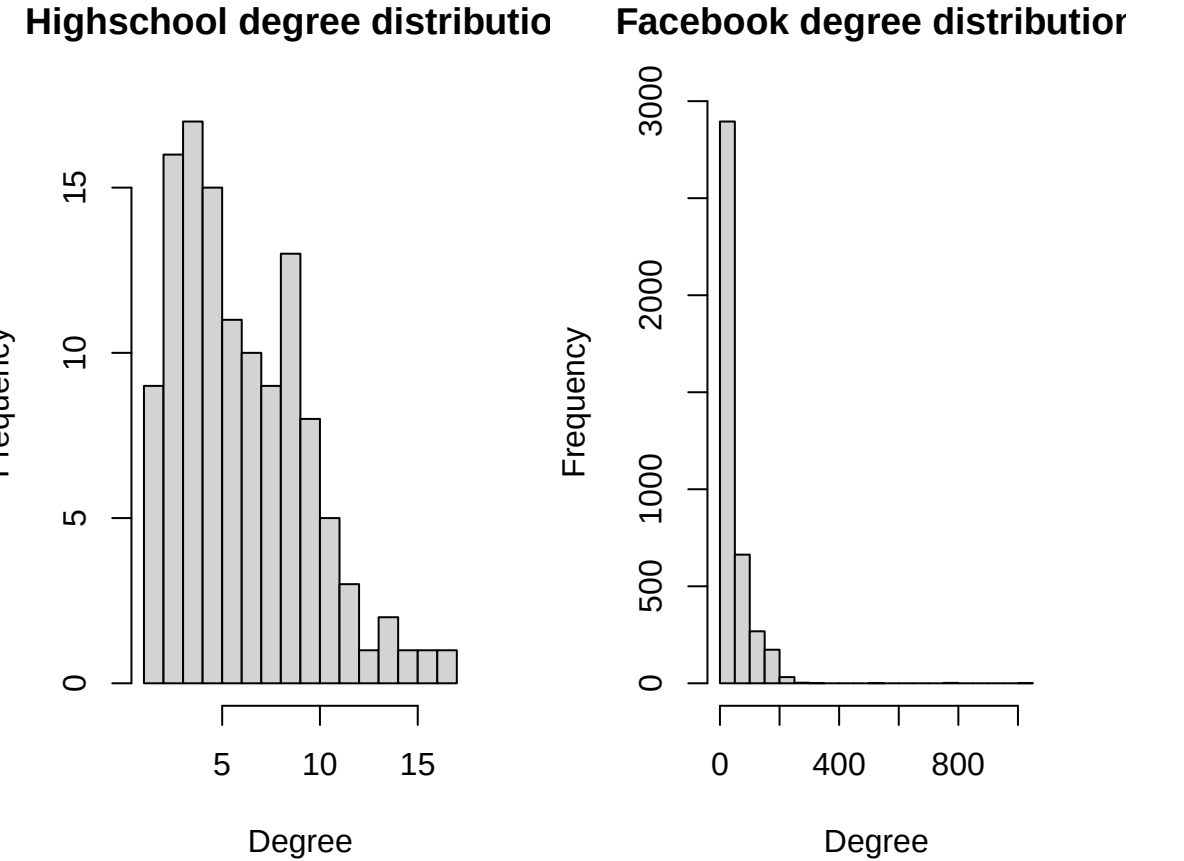
hist(deg_high,
      breaks=20,
      main="Highschool degree distribution",
      xlab="Degree")

hist(deg_fb,
      breaks=20,
      main="Facebook degree distribution",
      xlab="Degree")
```



(Note: the vertical reference line shows the values of shortest path length or the degree at certain percentiles.)

Figure 1: One important stand of community detection algorithm is based on **modularity**, which tries to maximize the difference between the actual number of edges in a community and the expected number of edges in the community. However, optimizing modularity in a network is NP-hard, therefore have to use heuristics.



Degree distribution and shortest path lengths

The two networks have quite different degree distributions. In the Highschool network, the distribution is relatively narrow and moderately right-skewed: most nodes have between about 3 and 8 connections, and the highest degrees are around 15–17. Those nodes have more connections than others but do not dominate the network. The Facebook network, by contrast, is highly right-skewed: most nodes have few connections, while a small number have very high degrees (around 200). Those highly connected nodes act as strong hubs.

This difference helps explain the path lengths we saw above. In the Facebook network, high-degree hubs create shortcuts across the network, so that many otherwise distant pairs are only a few steps apart. In the Highschool network, degrees are more evenly spread and hubs are weaker, so paths depend more on chains of intermediate nodes. Because that network is also small, most pairs are still reachable within six steps; in the larger Facebook network, the presence of hubs does a lot of the work in keeping typical path lengths short.

Question 2 (5 points) *(please submit your code along with your answer)*

- 1) Calculate the modularity of the Highschool network if community is merely identified by a) gender and b) residential hall, respectively.
- 2) Use the Louvain Community Detection to identify communities in the Highschool network. Compare the modularity value produced by the Louvain algorithm to those in 1), and explain the reasons for the differences.

```
# community by gender
genderCommunity <- Highschool_nodes$gender
genderCommunity <- replace(genderCommunity, genderCommunity == "female", 1)
genderCommunity <- replace(genderCommunity, genderCommunity == "male", 2)
genderCommunity <- replace(genderCommunity, genderCommunity == "unknown", 3)
genderCommunity <- as.numeric(genderCommunity)

gender.clustering <- make_clusters(Highschool, membership = genderCommunity)
mod_gender <- modularity(gender.clustering)
```

```
#community by residential hall
hallCommunity <- as.factor(Highschool_nodes$hall)
hallCommunity <- as.numeric(hallCommunity)

hall.clustering <- make_clusters(Highschool, membership = hallCommunity)
mod_hall <- modularity(hall.clustering)
```

```
#Louvain algorithm
Louv <- cluster_louvain(Highschool)
mod_louv <- modularity(Louv)
```

```
data.frame(
  method = c("Gender", "Residential Hall", "Louvain"),
  modularity = c(mod_gender, mod_hall, mod_louv)
)
```

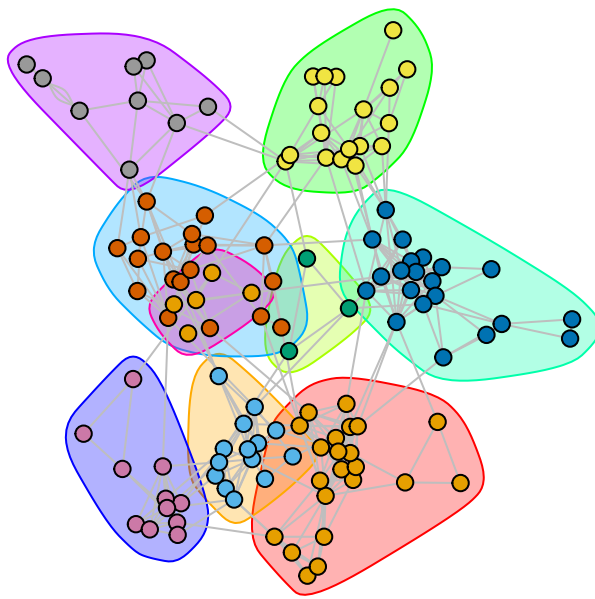
```
##           method  modularity
```



```
## 1          Gender 4.782675e-05
## 2 Residential Hall 1.755911e-01
## 3          Louvain 7.016599e-01
```

```
plot(
  Louv,
  Highschool,
  layout = layout_with_fr(Highschool),
  vertex.size = 6,
  vertex.label = NA,
  edge.color = "gray",
  mark.groups = communities(Louv),
  main = "Louvain communities in the Highschool network"
)
```

Louvain communities in the Highschool network



Answer

We compared three ways of defining communities on the Highschool network: by gender, by residential hall, and using the Louvain algorithm. The modularity values tell a clear story.

When we partition the network by gender, modularity is extremely low (about 0.00005). Friendships in this network are not organised along gender lines; male and female students are often connected to each other

rather than forming separate groups. When we partition by residential hall, modularity is higher (about 0.176). Students are somewhat more likely to be friends with others in the same hall, so hall membership captures part of the structure—but only part, and the value remains moderate.

The Louvain algorithm does much better, with modularity around 0.701. The reason is that gender and hall are fixed attributes we impose on the network, whereas Louvain groups nodes directly from their connection patterns and thus optimizes modularity. The communities it finds therefore align much more closely with how ties are actually distributed. In the plot we see densely connected clusters with relatively few links between them—likely friend circles, shared activities, or overlapping groups that are not neatly summarized by a single attribute like gender or hall. So in short: gender explains almost none of the structure, hall explains some of it, and Louvain recovers the strongest community structure because it uses the topology of the network itself.

Exercise 2

You might realize not every value of p can return you a small-world network that you are looking for. Then a question arises as how can one find the range of p . In the Figure 2 of Watts and Strogatz (1998), it explains how can one decide the range of p by looking at the dynamics between path length and clustering coefficient.

Question 3 (4 points)

- 1) Start with a regular network of size=300, nei=6, first reproduce the Figure 2 of Watts and Strogatz (1998). Then provide the range of p which can turn this regular network (size=300, nei=6) into a small-world network.
- 2) Do you need to rewire significant amount of connections to make the network small-world-like?
- 3) In the paper of Watts and Strogatz (1998), they pointed out that the value of p has two important implications:
 - “The idealized construction above reveals the key role of short cuts. It suggests that the small-world phenomenon might be common in sparse networks with many vertices, as even a tiny fraction of short cuts would suffice.”
 - “Thus, infectious diseases are predicted to spread much more easily and quickly in a small world; the alarming and less obvious point is how few short cuts are needed to make the world small.”

Use your own words to explain these two implications. For the second implication, connect it with the spread of COVID.

Answer

```
set.seed(123)

# Regular network: size = 300, total degree = 6, so nei = 3 on each side
g_regular <- sample_smallworld(dim = 1, size = 300, nei = 3, p = 0)

C0 <- transitivity(g_regular, type = "global")
L0 <- mean_distance(g_regular, directed = FALSE)

p_values <- c(0, 10^seq(-4, 0, length.out = 20))
```

```

results_q3 <- lapply(p_values, function(p) {
  g <- sample_smallworld(dim = 1, size = 300, nei = 3, p = p)

  Cp <- transitivity(g, type = "global")
  Lp <- mean_distance(g, directed = FALSE)

  data.frame(
    p = p,
    clustering = Cp,
    path_length = Lp,
    clustering_norm = Cp / C0,
    path_length_norm = Lp / L0
  )
}) %>%
  bind_rows()

results_q3

```

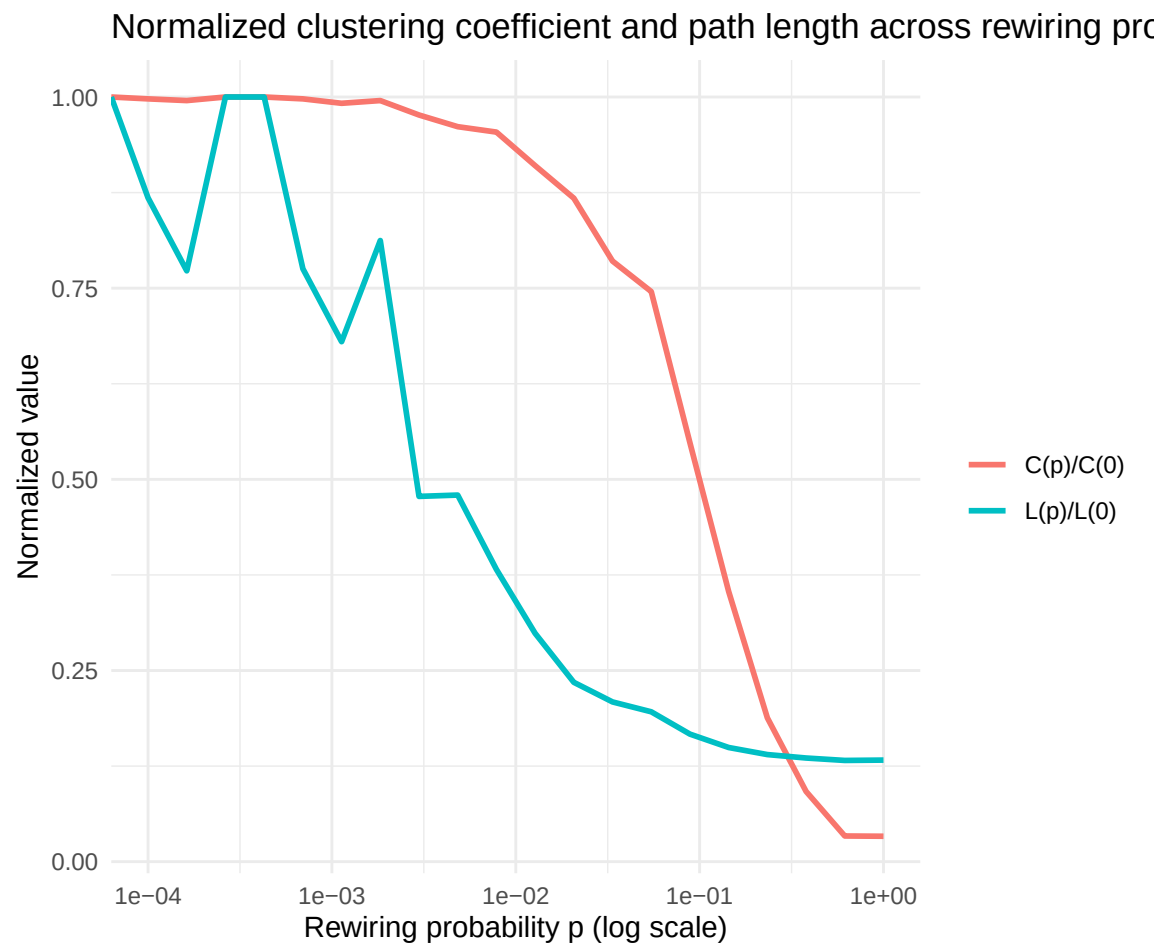
##		p	clustering	path_length	clustering_norm	path_length_norm
## 1	0.0000000000	0.60000000	25.418060	1.00000000	1.00000000	
## 2	0.0001000000	0.59853366	22.066667	0.99755610	0.8681491	
## 3	0.0001623777	0.59720062	19.639844	0.99533437	0.7726728	
## 4	0.0002636651	0.60000000	25.418060	1.00000000	1.00000000	
## 5	0.0004281332	0.60000000	25.418060	1.00000000	1.00000000	
## 6	0.0006951928	0.59853366	19.709164	0.99755610	0.7754000	
## 7	0.0011288379	0.59506886	17.284348	0.99178143	0.6800026	
## 8	0.0018329807	0.59720062	20.648004	0.99533437	0.8123360	
## 9	0.0029763514	0.58588549	12.142631	0.97647581	0.4777167	
## 10	0.0048329302	0.57671840	12.185730	0.96119734	0.4794123	
## 11	0.0078475997	0.57247340	9.714916	0.95412234	0.3822053	
## 12	0.0127427499	0.54617764	7.590569	0.91029607	0.2986289	
## 13	0.0206913808	0.52059018	5.961449	0.86765030	0.2345360	
## 14	0.0335981829	0.47125932	5.309967	0.78543221	0.2089053	
## 15	0.0545559478	0.44735121	4.981717	0.74558535	0.1959912	
## 16	0.0885866790	0.32865169	4.240401	0.54775281	0.1668263	
## 17	0.1438449888	0.21200084	3.795452	0.35333473	0.1493211	
## 18	0.2335721469	0.11286036	3.559777	0.18810059	0.1400491	
## 19	0.3792690191	0.05512573	3.445641	0.09187621	0.1355588	
## 20	0.6158482111	0.02014968	3.365580	0.03358281	0.1324090	
## 21	1.0000000000	0.01995060	3.376737	0.03325100	0.1328479	

```

ggplot(results_q3, aes(x = p)) +
  geom_line(aes(y = clustering_norm, color = "C(p)/C(0)"), linewidth = 1) +
  geom_line(aes(y = path_length_norm, color = "L(p)/L(0)"), linewidth = 1) +
  scale_x_log10() +
  labs(
    title = "Normalized clustering coefficient and path length across rewiring probabilities",
    x = "Rewiring probability p (log scale)",
    y = "Normalized value",
    color = ""
  ) +
  theme_minimal()

```

```
## Warning in scale_x_log10(): log-10 transformation introduced infinite values.
## log-10 transformation introduced infinite values.
```



```
small_world_range <- results_q3 %>%
  filter(path_length_norm < 0.5, clustering_norm > 0.5)

small_world_range
```

```
##           p clustering path_length clustering_norm path_length_norm
## 1 0.002976351 0.5858855 12.142631      0.9764758      0.4777167
## 2 0.004832930 0.5767184 12.185730      0.9611973      0.4794123
## 3 0.007847600 0.5724734  9.714916      0.9541223      0.3822053
## 4 0.012742750 0.5461776  7.590569      0.9102961      0.2986289
## 5 0.020691381 0.5205902  5.961449      0.8676503      0.2345360
## 6 0.033598183 0.4712593  5.309967      0.7854322      0.2089053
## 7 0.054555948 0.4473512  4.981717      0.7455853      0.1959912
## 8 0.088586679 0.3286517  4.240401      0.5477528      0.1668263
```

```
range(small_world_range$p)
```

```
## [1] 0.002976351 0.088586679
```

To reproduce Figure 2 from Watts and Strogatz (1998), we generated a regular network with size = 300 and degree = 6, and gradually increased the rewiring probability p . For each value of p , we calculated the clustering coefficient and average path length and normalized both relative to the original regular network. This allows us to observe how local clustering and global distances change as shortcuts are added.

The results show that the normalized average path length decreases quickly even for very small values of p , while the normalized clustering coefficient remains high over a wider range. This means that only a few shortcut connections are enough to make nodes much closer to each other without strongly changing the local structure. This pattern matches the behaviour of small-world networks described by Watts and Strogatz (1998).

Based on our simulation, the small-world region appears approximately between $p = 0.003$ and $p = 0.089$. In this range, clustering is still relatively high while path length is already much shorter than in the regular network.

These results show that only limited rewiring is needed to create a small-world network. A small number of long-range connections can strongly reduce global distances while keeping local neighbourhood structure mostly unchanged.

One implication is that the small-world property can easily appear in many real-world networks without requiring the network to become random. Even a few bridging ties can connect distant parts of the network efficiently.

Another implication is that spreading processes become faster in small-world networks. Shortcut connections link otherwise distant groups, allowing diseases, information, or behaviours to spread quickly. For example, during COVID-19, long-distance travel and large gatherings acted as shortcut connections between communities and accelerated transmission across regions.

Overall, the results confirm that small-world structure emerges at intermediate rewiring levels, consistent with Watts and Strogatz (1998).

Exercise 3

Question 4 (8 points)

Assume you want to promote a vegetarian recipe in the high school network (same as the “Highschool_network_att.csv” and “Highschool_network_att.csv” used in Question 1).

Each student in the network has an adoption threshold, which is listed in “Highschool_network_edge.csv” under the column Threshold. A student adopts the recipe only when the proportion of their friends who have already adopted it reaches or exceeds their threshold. For example, student S1 has a threshold of 0.51. This means S1 will adopt the vegetarian recipe only if at least 51% of her friends have adopted it; otherwise, she remains inactive.

Your goal is to identify the best initial seed set for promoting the vegetarian recipe in this network.

Assume you have a budget to seed 5 students initially: 1) Use greedy algorithm to find out 5 seeds and provide their IDs; 2) Use degree heuristics to find out 5 seeds and provide their IDs; 3) Use betweenness heuristics to find out 5 seeds and provide their IDs; 4) Compare the contagion outcome, discuss which strategy works the best in this network and why.

Answer

```
threshold <- Highschool_att$Threshold
```

```
calculate_adoptedNei <- function(node, node_status, each_neighbors) {
  mean(node_status[each_neighbors[[node]]] == 1)
}
```

```
ThModel <- function(node_seed, network, threshold) {

  adj_matrix <- as.matrix(as_adjacency_matrix(network, type = "both"))

  each_neighbors <- which(adj_matrix > 0, arr.ind = TRUE)
  each_neighbors <- split(each_neighbors[,2], each_neighbors[,1])

  nNode <- vcount(network)

  node_status <- rep(0, nNode)
  neighbour_status <- rep(0, nNode)

  new_infected <- list()
  day_total_infected <- rep(0, 28)

  # Day 1
  day <- 1
  node_status[node_seed] <- 1

  new_infected[[day]] <- node_seed
  day_total_infected[day] <- sum(node_status)

  # Days 2-28
  for (day in 2:28) {

    NotAdopted <- which(node_status == 0)
    Adopted <- which(node_status == 1)

    neighbour_status[NotAdopted] <- unlist(
      lapply(NotAdopted, calculate_adoptedNei,
             node_status, each_neighbors)
    )

    new_infected[[day]] <- setdiff(
      which(neighbour_status >= threshold),
      Adopted
    )

    node_status[new_infected[[day]]] <- 1

    day_total_infected[day] <- sum(node_status)
  }

  return(list(day_total_infected, new_infected))
}
```

```
greedy_ThM <- function(network, threshold, k) {

  nNode <- vcount(network)
```

```

SeedCan <- 1:nNode
Seedset <- c()

for (s in 1:k) {

  best_seed <- -1
  best_spread <- -Inf

  for (candidate in SeedCan) {

    trial_set <- c(Seedset, candidate)
    spread <- ThModel(trial_set, network, threshold)[[1]]

    if (sum(spread) > sum(best_spread)) {
      best_seed <- candidate
      best_spread <- spread
    }
  }

  Seedset <- c(Seedset, best_seed)
  SeedCan <- setdiff(SeedCan, Seedset)
}

Seedset
}

```

```

greedy_ThM5 <- greedy_ThM(Highschool, threshold, 5)

V(Highschool)$name[greedy_ThM5]

```

```
## [1] "S77" "S99" "S92" "S20" "S119"
```

```

ThM_greedy <- ThModel(greedy_ThM5, Highschool, threshold)

final_greedy_active <- max(ThM_greedy[[1]])
final_greedy_active

```

```
## [1] 65
```

```

deg <- degree(Highschool)

degree_5 <- order(deg, decreasing = TRUE)[1:5]

V(Highschool)$name[degree_5]

```

```
## [1] "S54" "S20" "S110" "S64" "S96"
```

```

ThM_degree <- ThModel(degree_5, Highschool, threshold)

final_degree_active <- max(ThM_degree[[1]])
final_degree_active

```

```
## [1] 18
```

```
btw <- betweenness(Highschool, directed = FALSE, normalized = TRUE)

betweenness_5 <- order(btw, decreasing = TRUE)[1:5]

V(Highschool)$name[betweenness_5]
```

```
## [1] "S37" "S4" "S96" "S24" "S70"
```

```
ThM_betweenness <- ThModel(betweenness_5, Highschool, threshold)

final_betweenness_active <- max(ThM_betweenness[[1]])
final_betweenness_active
```

```
## [1] 34
```

```
comparison_q4 <- data.frame(
  method = c("Greedy", "Degree heuristic", "Betweenness heuristic"),
  seeds = c(
    paste(V(Highschool)$name[greedy_ThM5], collapse = ", "),
    paste(V(Highschool)$name[degree_5], collapse = ", "),
    paste(V(Highschool)$name[betweenness_5], collapse = ", ")
  ),
  activated_nodes = c(
    final_greedy_active,
    final_degree_active,
    final_betweenness_active
  )
)

comparison_q4
```

```
##               method               seeds activated_nodes
## 1           Greedy S77, S99, S92, S20, S119             65
## 2 Degree heuristic S54, S20, S110, S64, S96             18
## 3 Betweenness heuristic S37, S4, S96, S24, S70           34
```

```
library(tidyr)
```

```
##
## Attaching package: 'tidyr'
```

```
## The following object is masked from 'package:igraph':
##
## crossing
```

```
history_df <- data.frame(
  Day = 1:28,
  Greedy = ThM_greedy[[1]],
```



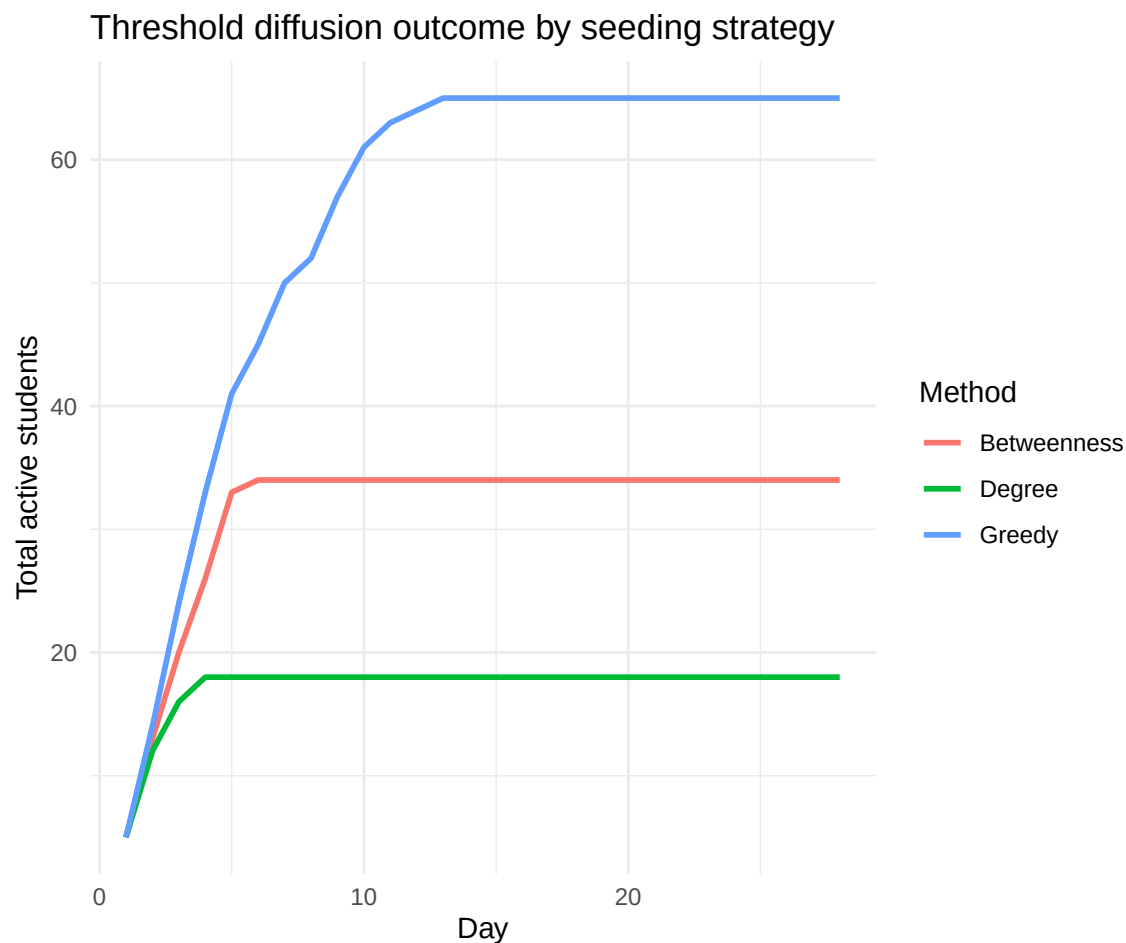
```

Degree = ThM_degree[[1]],
Betweenness = ThM_betweenness[[1]]
)

history_long <- pivot_longer(
  history_df,
  cols = c("Greedy", "Degree", "Betweenness"),
  names_to = "Method",
  values_to = "TotalActive"
)

ggplot(history_long,
  aes(x = Day, y = TotalActive, color = Method)) +
  geom_line(linewidth = 1) +
  labs(
    title = "Threshold diffusion outcome by seeding strategy",
    x = "Day",
    y = "Total active students"
  ) +
  theme_minimal()

```



We analysed the spread of a vegetarian recipe in the Highschool network using a threshold model. In this model, a student adopts the recipe only when the proportion of their friends who already adopted it

reaches their threshold. This means adoption depends on reinforcement from multiple neighbours (complex contagion).

1) Greedy algorithm seeds

The greedy algorithm selected the following 5 seeds:

S77, S99, S92, S20, S119

This strategy activated 65 students in total.

2) Degree heuristic seeds

The degree heuristic selected:

S54, S20, S110, S64, S96

This strategy activated 18 students.

3) Betweenness heuristic seeds

The betweenness heuristic selected:

S37, S4, S96, S24, S70

This strategy activated 34 students.

4) Comparison of strategies

The greedy strategy clearly performs best:

Greedy: 65 activated students Betweenness: 34 activated students Degree: 18 activated students

The plot also shows that the greedy strategy continues spreading for more steps and reaches the highest final adoption level.

The greedy algorithm works best because it selects seeds that increase the spread step by step across the network. In threshold diffusion, this is important since adoption requires reinforcement from several neighbours.

The degree heuristic performs worst. High-degree students have many connections, but they may be located close to each other, so their influence overlaps and does not spread widely across the network.

The betweenness heuristic performs better than degree because it selects students who connect different parts of the network. However, it still does not consider how seeds work together as a group.

Overall, the greedy algorithm produces the largest cascade because it selects seeds that are better distributed across the network and create stronger reinforcement effects.

Question 5 (8 points)

The seeding strategies in Question 4 assume access to complete network information. In real-world settings, however, such information is often unavailable because collecting full network data is costly and difficult. Without knowing the exact degree, betweenness, or other centrality measures of each node, it is not possible to directly implement strategies such as degree-based heuristics, betweenness-based heuristics, or greedy seeding.

To address this constraint, researchers have proposed practical seeding methods inspired by the Friendship Paradox. One such method is the one-hop strategy: first sample individuals uniformly at random from the network, then ask each sampled individual to nominate one person they know, and finally use the nominated individuals as the initial seeds.

Answer the following questions:

- 1) Explain how the one-hop strategy is related to the Friendship Paradox.
- 2) Implement the one-hop strategy on the high school network to identify 5 seeds. You will observe that the resulting seed set may vary from run to run. Explain the reason for this variation.
- 3) Since the one-hop seed set is not fixed, propose an appropriate approach to compare its contagion performance against a random seeding strategy.
- 4) Implement the approach proposed in part (3) and compare the contagion outcomes of the two strategies in this network.
- 5) Conclude the strengths and limitations of the one-hop strategy.

Answer

```
one_hop_seed <- function(network, k = 5) {  
  
  nNode <- vcount(network)  
  final_seeds <- c()  
  
  while (length(final_seeds) < k) {  
  
    # sample one student uniformly at random  
    sampled_node <- sample(1:nNode, 1)  
  
    # get neighbors of sampled student  
    neigh <- neighbors(network, sampled_node)  
  
    # only proceed if the sampled node has at least one neighbor  
    if (length(neigh) > 0) {  
      nominated <- sample(neigh, 1)  
      final_seeds <- unique(c(final_seeds, nominated))  
    }  
  }  
  
  final_seeds[1:k]  
}
```

```
random_seed <- function(network, k = 5) {  
  sample(1:vcount(network), k, replace = FALSE)  
}
```

```
set.seed(123)
```

```
onehop_5 <- one_hop_seed(Highschool, 5)
```

```
onehop_5
```

```
## [1] 37 95 34 116 73
```

```
V(Highschool)$name[onehop_5]
```

```
## [1] "S37" "S95" "S34" "S116" "S73"
```

```
ThM_onehop <- ThModel(onehop_5, Highschool, threshold)
```

```
final_onehop_active <- max(ThM_onehop[[1]])  
final_onehop_active
```

```
## [1] 13
```

```
set.seed(1)  
V(Highschool)$name[one_hop_seed(Highschool, 5)]
```

```
## [1] "S120" "S1" "S17" "S52" "S64"
```

```
set.seed(2)  
V(Highschool)$name[one_hop_seed(Highschool, 5)]
```

```
## [1] "S81" "S94" "S3" "S28" "S90"
```

```
set.seed(3)  
V(Highschool)$name[one_hop_seed(Highschool, 5)]
```

```
## [1] "S92" "S32" "S73" "S91" "S15"
```

```
evaluate_strategy <- function(network, threshold, strategy_fun, k = 5, n_sim = 200) {  
  
  results <- data.frame(  
    run = 1:n_sim,  
    activated_nodes = numeric(n_sim)  
  )  
  
  seed_list <- vector("list", n_sim)  
  
  for (i in 1:n_sim) {  
    seeds <- strategy_fun(network, k)  
    seed_list[[i]] <- V(network)$name[seeds]  
  
    outcome <- ThModel(seeds, network, threshold)  
    results$activated_nodes[i] <- max(outcome[[1]])  
  }  
  
  list(  
    results = results,  
    seeds = seed_list  
  )  
}
```

```

set.seed(123)

onehop_eval <- evaluate_strategy(
  network = Highschool,
  threshold = threshold,
  strategy_fun = one_hop_seed,
  k = 5,
  n_sim = 200
)

random_eval <- evaluate_strategy(
  network = Highschool,
  threshold = threshold,
  strategy_fun = random_seed,
  k = 5,
  n_sim = 200
)

```

```

summary_comparison_q5 <- data.frame(
  method = c("One-hop", "Random"),
  mean_activated = c(
    mean(onehop_eval$results$activated_nodes),
    mean(random_eval$results$activated_nodes)
  ),
  median_activated = c(
    median(onehop_eval$results$activated_nodes),
    median(random_eval$results$activated_nodes)
  ),
  sd_activated = c(
    sd(onehop_eval$results$activated_nodes),
    sd(random_eval$results$activated_nodes)
  ),
  min_activated = c(
    min(onehop_eval$results$activated_nodes),
    min(random_eval$results$activated_nodes)
  ),
  max_activated = c(
    max(onehop_eval$results$activated_nodes),
    max(random_eval$results$activated_nodes)
  )
)

```

```
summary_comparison_q5
```

```

##      method mean_activated median_activated sd_activated min_activated
## 1 One-hop          19.88              18      8.410666              6
## 2 Random           18.15              16      9.013530              5
##      max_activated
## 1                60
## 2                57

```

```

t.test(
  onehop_eval$results$activated_nodes,
  random_eval$results$activated_nodes
)

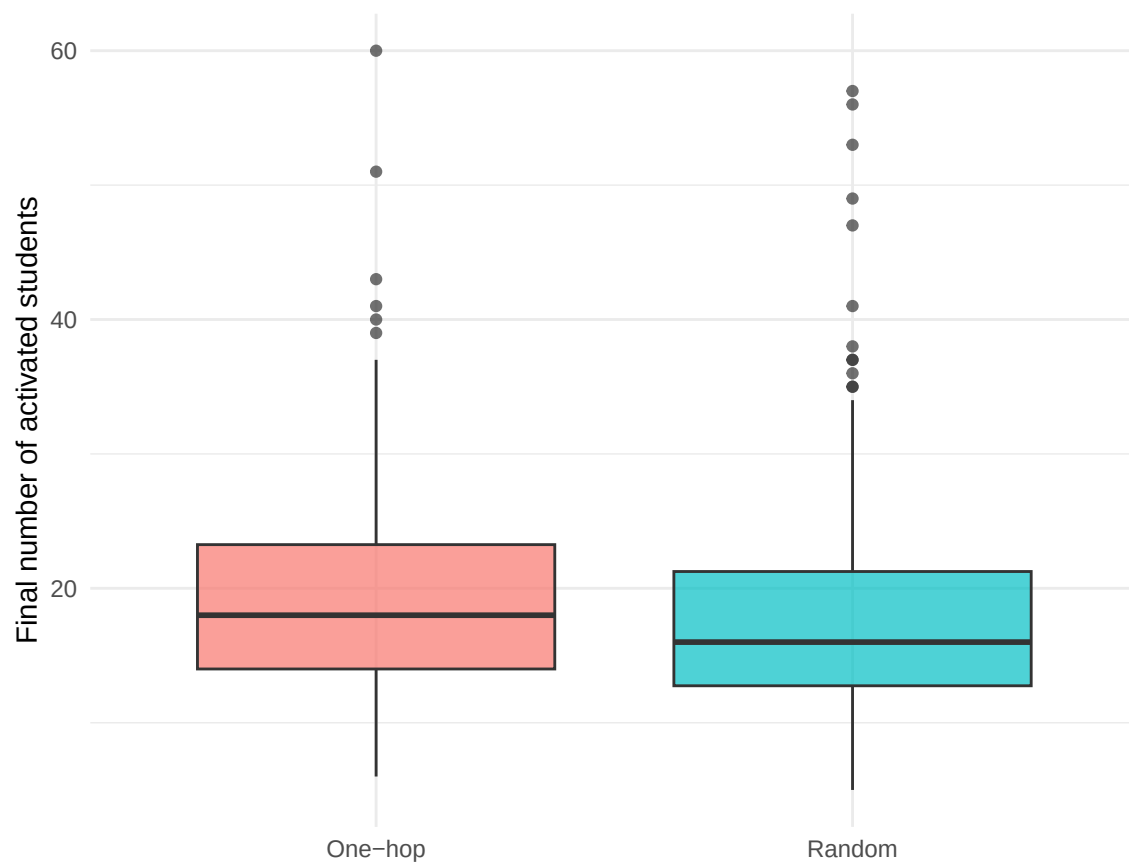
##
## Welch Two Sample t-test
##
## data: onehop_eval$results$activated_nodes and random_eval$results$activated_nodes
## t = 1.9846, df = 396.11, p-value = 0.04788
## alternative hypothesis: true difference in means is not equal to 0
## 95 percent confidence interval:
##  0.01620201 3.44379799
## sample estimates:
## mean of x mean of y
##      19.88      18.15

comparison_plot_q5 <- rbind(
  data.frame(method = "One-hop", activated_nodes = onehop_eval$results$activated_nodes),
  data.frame(method = "Random", activated_nodes = random_eval$results$activated_nodes)
)

ggplot(comparison_plot_q5, aes(x = method, y = activated_nodes, fill = method)) +
  geom_boxplot(alpha = 0.7) +
  labs(
    title = "Threshold diffusion outcome: one-hop vs random seeding",
    x = "",
    y = "Final number of activated students"
  ) +
  theme_minimal() +
  theme(legend.position = "none")

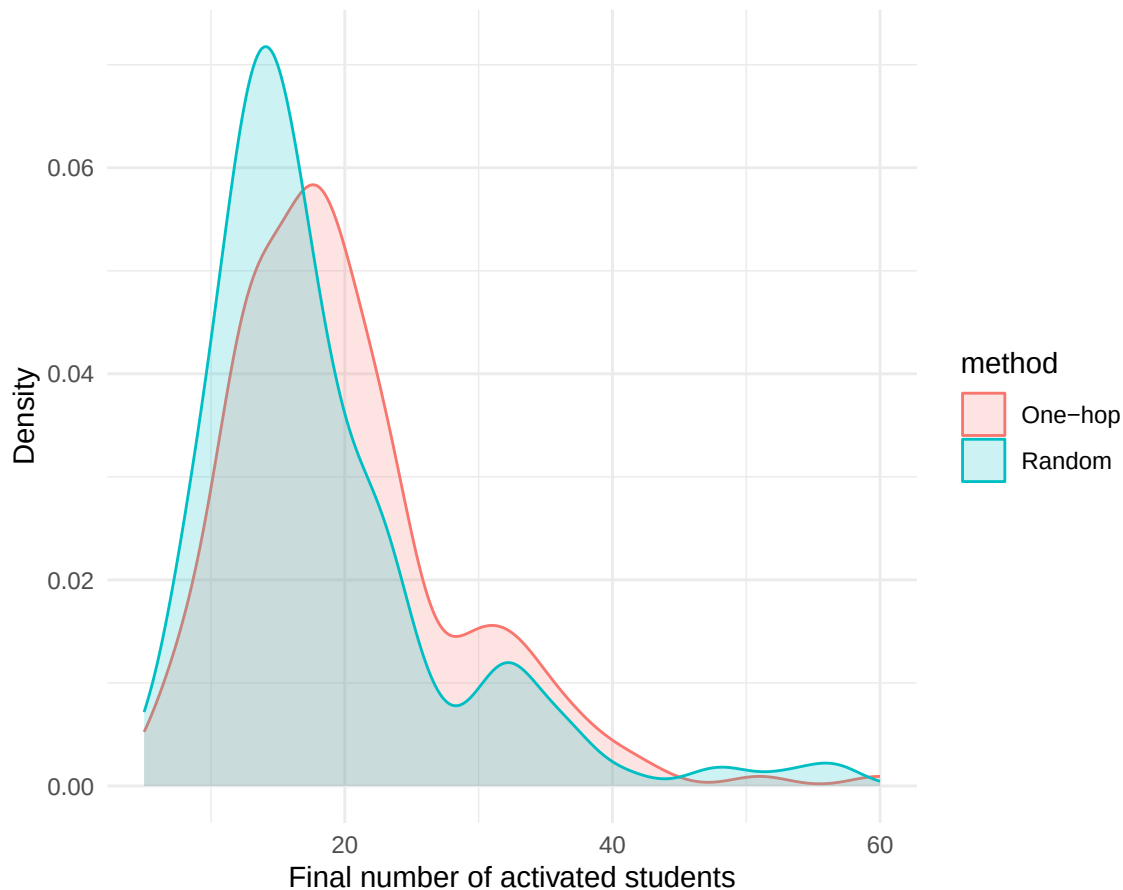
```

Threshold diffusion outcome: one-hop vs random seeding



```
ggplot(comparison_plot_q5, aes(x = activated_nodes, color = method, fill = method)) +  
  geom_density(alpha = 0.2) +  
  labs(  
    title = "Distribution of final activated students",  
    x = "Final number of activated students",  
    y = "Density"  
  ) +  
  theme_minimal()
```

Distribution of final activated students



```
example_onehop_seeds <- V(Highschool)$name[onehop_5]
example_onehop_seeds
```

```
## [1] "S37" "S95" "S34" "S116" "S73"
```

The one-hop strategy is based on the Friendship Paradox, which states that your friends usually have more friends than you do. In the one-hop strategy, we first select students randomly and then ask each of them to nominate one friend. Since highly connected students appear more often in friendship lists, they are more likely to be selected as seeds than with pure random sampling.

Using the one-hop strategy, one example seed set was:

S37, S95, S34, S116, S73

This seed set activated 13 students in that run.

However, the selected seeds change across runs because the strategy includes randomness both when choosing initial students and when selecting their nominated friends. Therefore, different runs produce different contagion outcomes.

To compare one-hop with random seeding fairly, both strategies were repeated many times and their average performance was compared instead of relying on a single run.

We repeated both strategies 200 times. The one-hop strategy activated on average 19.88 students, while random seeding activated 18.15 students. The median values were 18 and 16 respectively. The maximum spread was also slightly higher for one-hop (60 vs 57).

A Welch two-sample t-test gave a p-value of 0.04788, which shows that the difference between the two strategies is statistically significant at the 5% level. This means one-hop performs slightly better than random seeding in this network.

The advantage of the one-hop strategy is that it does not require full network information but still tends to select better-connected students. This makes it useful when centrality measures cannot be calculated.

However, the strategy still depends on randomness and does not optimise seed placement like the greedy algorithm. In Question 4, greedy seeding activated 65 students, which is much higher than the one-hop average. This shows that while one-hop improves over random selection, it is less effective than strategies that use complete network structure.

Overall, the one-hop strategy is a practical method when network data is limited. It performs better than random seeding on average, but it cannot match the performance of greedy seeding in threshold diffusion settings.